Assignment 3

Tejas Gupta
B22CS093

February 22, 2026

1 Model Selection

The model chosen for this task is **DistilBERT** (`distilbert-base-cased`), a distilled version of BERT developed by Hugging Face. DistilBERT was selected for the following reasons:

- It retains 97% of BERT's language understanding capability while being 60% faster and 40% smaller in size.
- The *cased* variant preserves capitalization, which can carry useful information for distinguishing genres (e.g., poetry vs. comics).
- It integrates seamlessly with the Hugging Face `Trainer` API, simplifying the training and evaluation pipeline.
- Its smaller footprint makes it practical for training in resource-constrained environments such as Docker containers and CPU-only machines.

The task is multi-class text classification: predicting the genre of a Goodreads book review from among 8 categories — poetry, children, comics & graphic, fantasy & paranormal, history & biography, mystery/thriller/crime, romance, and young adult.

2 Training Summary

The dataset consists of Goodreads book reviews sourced from the UCSD Book Graph. Reviews were streamed from compressed JSON files, sampled per genre, and split into training and test sets.

The training loss decreased from 1.92 (epoch 1) to 1.32 (epoch 2), confirming that the model was learning meaningful patterns from the data. Evaluation was performed at every 100 training steps to monitor progress.

The notebook was converted into modular Python scripts:

- `data.py` — Data loading, tokenization, and dataset preparation
- `utils.py` — Metrics computation and result saving
- `train.py` — Training with Hugging Face Trainer API
- `eval.py` — Evaluation from local model or HuggingFace Hub

3 Evaluation Comparison

After training, the model was evaluated locally, then pushed to Hugging Face Hub (`Tron2703/distilbert-goodreads-genre`). It was then re-loaded from the Hub and

Parameter	Value
Pre-trained model	`distilbert-base-cased`
Number of classes	8
Training samples	1600 (200 per genre)
Test samples	400 (50 per genre)
Epochs	2
Batch size (train / eval)	16 / 16
Learning rate	5e-5
Warmup steps	50
Weight decay	0.01
Max token length	512

Table 1: Training configuration

evaluated again to verify consistency.

Source	Accuracy	F1 (weighted)	Loss
DistilBERT (local)	0.470	0.445	1.478
DistilBERT (from HF Hub)	0.518	0.508	1.363
Original notebook (GPU, full data)	$\sim$0.590	$\sim$0.590	$\sim$1.280

Table 2: Evaluation results comparison

The slight difference between local and Hub evaluations is due to random sampling of test data in each run (the data is re-downloaded and re-sampled each time). Both results are well above the random baseline of 12.5% for 8 classes.

The gap from the original notebook's $\sim$59% accuracy is attributable to the reduced training set (1600 vs. 6400 samples) and fewer epochs (2 vs. 3). The code supports scaling up these parameters for improved accuracy when GPU resources are available.

4 Docker Setup

Two Docker images were created:

1. **Dockerfile** (Training): Based on `python:3.10-slim`, installs dependencies and runs `train.py`. Supports pushing the model to HuggingFace Hub via `-e HF_TOKEN` environment variable.
2. **Dockerfile.eval** (Production): Pulls the trained model from HuggingFace Hub and runs evaluation automatically on container startup. No training occurs inside this container.

The HuggingFace token is passed to Docker containers using the `-e` flag:

```
docker run -e HF_TOKEN=hf_xxxxx goodreads-train python train.py --push-to-hub
```

5 Challenges

1. **Training time on CPU:** The full dataset (6400 training samples, 3 epochs) requires approximately 12 hours on CPU. To make development feasible, the dataset

was reduced to 1600 training samples with 2 epochs, completing in ∼1.5 hours. DistilBERT's smaller architecture helped mitigate this compared to full BERT.

2. **Data download overhead:** The Goodreads review files are large compressed JSON archives hosted on UCSD servers. Streaming with `requests` and capping at 3000 reviews per genre kept memory usage manageable.

3. **Docker image size:** PyTorch and Transformers result in Docker images exceeding 2GB. Using `python:3.10-slim` as the base and `--no-cache-dir` for pip helped minimize the final image size.

4. **Accuracy vs. time tradeoff:** The reduced dataset yields 47–52% accuracy compared to ∼59% with full data on GPU. The training pipeline and code are identical; only data size differs. Scaling up the parameters in `data.py` will improve results given sufficient compute.

Links

- **HuggingFace Model:** `https://huggingface.co/Tron2703/distilbert-goodreads-genre`
- **GitHub Repository:** `https://github.com/TejasGupta-27/MLOps-TejasGupta-B22CS093/tree/Assignment3`